

Independent audit record

Repository source: `certificate/INDEPENDENT_AUDIT.md`

Contents

Scope	1
Audit matrix	1
Binary64 Python audit	3
Base-R audit	3
Reconstruction and regression tests	3
Adversarial mutation campaign	3
Construction-time export equivalence	4
Hosted release-candidate audit	4
Named-author remote fresh-clone audit	4
Audit-transfer integrity	5
Paper and documentation audits	5
Independence limitations	5
External audit still requested	5

Scope

This record describes the audit layers that are independent of the exact rational verification path to varying degrees. It does not call them independent mathematical review.

The exact verifier is the finite proof authority. The audits are intended to catch transcription, data-shape, arithmetic, assembly, portability, and implementation errors through separately organized computations.

Audited release-candidate commit: `892d4bc6fad07950e78da66e45b95063a6415af6` **Hosted workflow:** <https://github.com/swihart/minvol-degree3-contact-bound/actions/runs/35468019663>

Audit matrix

The records below use a vertical layout so that program paths, environments, and independence limits remain legible in both Markdown and PDF.

Exact verifier

- Program: `certificate/certify_universal_bound.py`
- Arithmetic or environment: Python integers and `Fraction`
- Relationship to the exact path: this is the authoritative proof path
- Current result: **PASS**

Binary64 reconstruction

- Program: `certificate/audit_universal_bound.py`
- Arithmetic or environment: NumPy binary64
- Relationship to the exact path: reads audit renderings and common immutable data
- Current result: **PASS locally and hosted**

Base-R reconstruction

- Program: `certificate/audit_universal_bound.R`
- Arithmetic or environment: base-R numerical arithmetic
- Relationship to the exact path: reads archived exact/decimal bridge data

- Current result: **PASS locally and hosted**

Reconstruction tests

- Program: `certificate/test_universal_bound.py`
- Arithmetic or environment: exact arithmetic and binary64
- Relationship to the exact path: calls production code and rebuilds proof objects
- Current result: **10/10 PASS**

Mutation tests

- Program: `certificate/test_adversarial_mutations.py`
- Arithmetic or environment: exact production gates
- Relationship to the exact path: operates on temporary mutated copies
- Current result: **7/7 rejected as required**

Construction-time export equivalence

- Record: `certificate/transcripts/export_equivalence.txt`
- Method: semantic field comparison
- Relationship to the exact path: compares the private source and public export at construction time
- Current result: **10 theorem-critical fields match**

Paper reconciliation

- Program: `paper/reconcile_certificate.py`
- Method: exact parsing and byte comparison
- Relationship to the exact path: reads the authoritative JSON and generated TeX
- Current result: **PASS**

Exact-constants appendix

- Program: `proof/generate_exact_constants.py`
- Method: deterministic JSON projection
- Relationship to the exact path: reads the authoritative JSON
- Current result: **CURRENT**

Release-document check

- Program: `ci/verify_release_docs.py`
- Method: structural and textual checks
- Relationship to the exact path: checks public repository documents
- Current result: **PASS**

Hosted candidate workflow

- Record: workflow run 35468019663
- Environment: Ubuntu 24.04.5; Python 3.13.15; R 4.6.1; Poppler
- Relationship to the exact path: executes project-supplied checks
- Current result: **3/3 jobs PASS**

Remote fresh-clone replay

- Record: named-author clone of commit 892d4bc6fad07950e78da66e45b95063a6415af6
- Environment: macOS; Python 3.14.7; NumPy 2.3.5; R 4.6.1
- Relationship to the exact path: uses the same repository objects and formulas in a new clone
- Current result: **full replay PASS; final status clean**

Remote-evidence index

- Record: `certificate/evidence/rc1_remote_audit.json`
- Method: structural provenance record
- Relationship to the exact path: summarizes the transferred logs and transcript
- Current result: **CURRENT**

Binary64 Python audit

`audit_universal_bound.py` is organized as a numerical reconstruction rather than a wrapper around the exact theorem function. It reports, among other quantities:

```
fan vertices:          98306
changed fan vertices:  18229
minimum binary64 fan margin: 2.888902546002714e-10
sharp determinant squared: 0.990112274299406
core volume lower:    0.411735710312777
three-pair cap lower: 0.000143960902055
near-Jung lower:      0.411879671214832
universal coefficient: 0.411877563780302
MINVOL UNIVERSAL CONTACT-BOUND BINARY64 AUDIT: PASS
```

Its agreement with the exact path is evidence against many ordinary implementation errors. It cannot certify a strict inequality whose margin is smaller than uncontrolled floating-point error, so it is not the proof authority.

Base-R audit

`audit_universal_bound.R` uses base R only and reconstructs the principal lower-row, fan, determinant, coordinate-width, cap, disjointness, and assembly quantities from the public audit records. Both the named-author fresh clone and the hosted R job end with:

```
MINVOL UNIVERSAL CONTACT-BOUND BASE-R AUDIT: PASS
```

Selected reported values include:

```
fan orbit rows / vertices: 24833/98306
minimum fan margin:       2.888904207092860e-10
determinant squared lower: 0.990112274299406
core volume lower:        0.411735710312777
three-pair cap lower:     0.000143960902055
near-Jung volume lower:   0.411879671214832
universal coefficient lower: 0.411877563780302
```

The archived construction transcript is `transcripts/base_r_audit.txt`. The named-author remote-clone replay used R 4.6.1. The hosted R job also installed and ran R 4.6.1. The assistant construction container does not include `Rscript`, so that language path is not represented as locally rerun there.

Reconstruction and regression tests

The ten Python tests cover:

- the binary64 audit;
- cap allocations and cross-axis disjointness;
- certificate claim fields;
- coordinate-width sharpening;
- byte-identical exact reconstruction;
- fan reconstruction and reference regression;
- reference-fan fingerprinting;
- gap-free refined lower rows;
- sharp determinant census; and
- absence of private-path dependencies.

The archived construction transcript is `transcripts/python_unittest.txt`. The same ten tests passed in the named-author fresh clone and the hosted Python job.

Adversarial mutation campaign

The mutation suite changes one theorem-critical object at a time in a temporary copy and requires rejection. The seven cases are:

Mutation	Intended gate
Lower-row endpoint	exact interval and propagation semantics
Final-fan radial numerator	fan geometry and margin semantics
Inherited S-procedure multiplier	exact multiplier consistency
Section-hull vertex	byte-identical hull reconstruction
Cap-allocation bracket	certificate reconstruction and cap semantics
Disjointness threshold	exact cross-axis separation gate
Theorem-coefficient digit	theorem-certificate byte identity

The archived construction transcript is `transcripts/adversarial_mutations.txt` and ends with:

```
MINVOL ADVERSARIAL MUTATION SUITE: PASS
```

The same seven rejection tests passed in the fresh clone and hosted Python job. These tests show that the selected failure paths are live. They are not a complete fault-injection proof for every line of code.

Construction-time export equivalence

Before the public-format certificate was separated from the private research repository, ten theorem-critical fields were compared semantically against the source package:

- theorem coefficient;
- split radius;
- lower-row list;
- bottleneck row;
- final fan minimum;
- forced-core lower bound;
- cap total;
- near-Jung lower bound;
- handoff margin; and
- relevant fan census/fingerprints.

The result is archived in `transcripts/export_equivalence.txt`. The public package does not require the private repository to replay.

Hosted release-candidate audit

The private staging repository workflow at <https://github.com/swihart/minvol-degree3-contact-bound/actions/runs/35468019663> checked out 892d4bc6fad07950e78da66e45b95063a6415af6 in every job. The named author reported all three jobs green. The downloaded log archive independently shows the expected pass markers and no error marker for:

1. exact certificate, binary64, ten tests, seven mutation tests, metadata, release documents, and paper reconciliation;
2. base-R reconstruction on R 4.6.1; and
3. the paper and 22-document build, pair checks, and Poppler preflight for both committed and locally rebuilt PDFs.

The hosted runner image was Ubuntu 24.04.5 LTS, image version 20260907.300.1. The Python job used CPython 3.13.15 and NumPy 2.3.5. The PDF job used Pandoc 3.10.1 and TeX Live 2023/Debian.

Named-author remote fresh-clone audit

The named-author transcript began at 2026-09-19 16:58:42 EDT from a fresh clone of the private staging remote. It records:

```
HEAD and origin/main: 892d4bc6fad07950e78da66e45b95063a6415af6
fresh-history commits: 5
Python: 3.14.7
NumPy: 2.3.5
```

```
R: 4.6.1
complete Python/R replay: PASS
paper build: PASS (10 pages)
committed Markdown/PDF pairs: 22/22 PASS
rebuilt Markdown/PDF pairs: 22/22 PASS
final git status --short: CLEAN
bundle complete history: PASS
```

The Mac did not run Poppler locally. That platform difference is covered by the successful hosted document job rather than hidden or weakened.

Audit-transfer integrity

The transferred evidence set was checked byte for byte. Its hashes are:

```
2c324230f02ba4cb2541e0d353cc7906ae460cac0a768267144f650a0278a1a3 minvol-degree3-contact-bound-v1.0.0-rc1
-main.bundle
ce2abd06646fdf89d6e60b9069ee55451715a35970afedd24e4acf0d1d64de7f minvol-degree3-contact-bound-v1.0.0-rc1
-fresh-replay.txt
44fc68b36e05153fabd2a93aaa4b092b946016b9eb10bcecf37d7bf5c6b4ad3a minvol-degree3-contact-bound-v1.0.0-rc1
-github-actions-logs.zip
```

The transfer manifest has SHA-256 3f958520e0e119cad4db97d60a2ac3702b8144a8f6228068bdfef6d43124f57d. The action-log ZIP contains 36 entries and passes archive-integrity testing. The machine-readable summary is [evidence/rc1_remote_audit.json](#).

Paper and documentation audits

`paper/reconcile_certificate.py` verifies that the manuscript uses generated certificate constants and that the lower-branch table is byte current. The Markdown/PDF pair checker requires exactly one committed PDF for every public Markdown source. The Poppler preflight checks readability, positive page count, font embedding, and extractable text.

Page-by-page visual inspection remains a human step. Automated PDF validity is not a mathematical proof check.

Independence limitations

The Python and R audits are independent implementations, but they are not fully independent experiments:

- they consume records derived from the same certificate objects;
- they implement the same mathematical formulas;
- they were prepared within the same project;
- the named-author fresh clone is not an outside reproduction;
- hosted CI executes project-supplied programs; and
- their agreement cannot rule out a common mathematical misconception.

Accordingly, the package should say **independent implementation audits**, not **independently verified theorem**.

External audit still requested

The most valuable next audit would be an external reviewer who independently:

1. checks the geometric reduction in the manuscript;
2. reimplements at least one critical exact component;
3. reconstructs a selected lower row or cap pair directly from the stored arrays;
4. verifies the branch handoff and universal quantifiers; and
5. reports any discrepancy publicly with the exact release tag and commit.